

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

по лабораторной работе
«Vue 3 (Vite): создание приложения заметок»

Выполнил:

Баранов Владимир

Группа

J3213

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Цель лабораторной работы состояла в том, чтобы освоить базовые команды пакетного менеджера `npm` и на практике запустить приложение на `Vue 3` с использованием `Vite`. В рамках задания требовалось не только создать фронтенд-проект, но и довести его до рабочего состояния: подключить маршрутизацию, выделить компоненты, настроить взаимодействие с API и реализовать страницу заметок с загрузкой и созданием записей.

Ход работы

Работа выполнялась последовательно: сначала был создан сам проект на `Vue 3`, затем подключены необходимые библиотеки, после чего были настроены роутинг, компоненты и хранилище. На завершающем этапе фронтенд был связан с `mock API`, а результат проверен в браузере.

1. Инициализация проекта и запуск окружения

На первом этапе приложение было создано через стандартный мастер `Vue` с поддержкой `Vite`. После генерации структуры проекта были установлены зависимости командой `npm install`. Дополнительно были настроены скрипты `package.json` так, чтобы команда `npm start` запускала локальный сервер разработки на порту `8080`.

2. Подключение библиотек и базовой инфраструктуры

Далее в проект были добавлены библиотеки `axios`, `bootstrap`, `pinia` и `pinia-persists`. На их основе был собран базовый инфраструктурный слой: создан экземпляр HTTP-клиента с `baseUrl`, настроены файлы для работы с API заметок, подключено централизованное хранилище `Pinia` и сохранение его состояния. В точке входа `main.js` были зарегистрированы `router` и `store`, а также подключены `Bootstrap`-стили, чтобы интерфейс сразу имел аккуратный базовый вид.

Дополнительно приложение было упрощено на уровне корневого компонента. В `App.vue` оставлен только `router-view`, что позволило передать

управление отображением страниц маршрутизатору и не перегружать корневую структуру лишней разметкой.

3. Настройка маршрутизации и компонентов

После подготовки инфраструктуры был настроен роутинг. Для главного маршрута был создан переход на страницу `NotesPage.vue`, которая загружается через `lazy import`. Такой подход соответствует компонентной архитектуре Vue и позволяет изолировать логику экрана в отдельном представлении.

Параллельно были созданы переиспользуемые компоненты. Базовый layout оформлен через `BaseLayout.vue` со slot, а отдельная карточка заметки вынесена в компонент `NoteCard.vue`. В результате каждая заметка отображается в едином формате, а основная страница остается чище и понятнее по структуре.

4. Реализация страницы заметок и хранилища Pinia

На странице `NotesPage.vue` была реализована форма с двухсторонним связыванием через `v-model` для полей `name` и `text`. Отправка формы обрабатывается методом `createCard`, который предотвращает стандартное поведение браузера, отправляет данные в API и затем обновляет список заметок. После успешного создания записи форма очищается, что делает сценарий взаимодействия завершенным и удобным для пользователя.

Для хранения данных было создано отдельное хранилище `notes.js`. В нем определено состояние `notes` и два основных действия: `loadNotes` для получения списка с сервера и `createNote` для отправки новой записи. На самой странице данные и методы подключаются через `mapState` и `mapActions`, а загрузка заметок вызывается в `mounted`. Благодаря этому после открытия страницы пользователь сразу видит актуальные данные, а новые заметки появляются без ручного обновления.

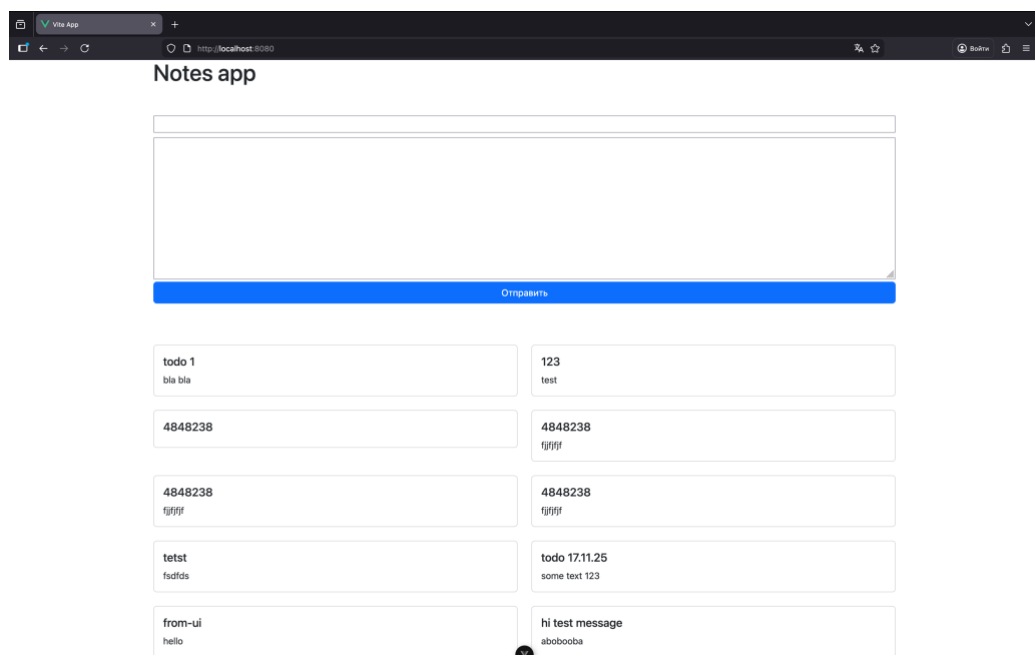
5. Подключение backend API и проверка обмена данными

Фронтенд был настроен на работу с API по адресу `http://localhost:3000`. Для этого использовались эндпоинты `GET /notes` и `POST /notes`. В качестве

серверной части была взята учебная заготовка на json-server из материалов мануала. Запуск backend выполнялся отдельно, после чего фронтенд получил возможность читать существующие заметки и добавлять новые записи через HTTP-запросы.

6. Проверка результата

После завершения настройки была проведена итоговая проверка приложения. При открытии стартовой страницы автоматически выполнялась загрузка заметок с backend API. Затем через форму создавалась новая запись, после чего список повторно отображался уже с добавленным элементом. Это подтвердило работоспособность всей связки: Vue-компоненты, Pinia, axios и mock API.



Вывод

В ходе работы был успешно создан учебный проект на Vue 3 с использованием Vite, а также закреплены базовые команды npm, необходимые для запуска и сопровождения фронтенд-приложения. В проект были подключены bootstrap, axios, pinia и pinia-persists, настроены маршрутизация и компонентная структура, а страница заметок получила полноценное взаимодействие с API.

Практический результат заключается в том, что приложение не только запускается локально, но и выполняет реальные пользовательские сценарии: получает данные с сервера, отображает их в виде карточек и позволяет создавать новые заметки через форму. Следовательно, цель лабораторной работы достигнута, а собранный проект можно рассматривать как корректный пример стартовой Vue-разработки с работой через API.